

TERRAFORM- CHALLENGES

1. Create an EC2 Instance on AWS Cloud using the below-mentioned ubuntu AMI and instance_type. Or, (You can use of your choice as well) ami = "ami-0461b1436c9b3c1a3" instance_type = "t2.micro"
2. Install a web server on the same EC2 Instance. The web server should listen on port 8080.
3. Create a Docker-compose file for the project:<https://github.com/betawins/cricbuzz-docker-hello-world.git> and point it to port 8000.
4. Make the web server point to the above service so that 8080 returns the response from the Docker service.
5. Configure to Save the Logs in the location /var/log/cb.log
6. Write the Log rotate configuration for the above file

Step 1: Configure Terraform Provider

The first step is to define the Terraform and AWS provider configuration.

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
}
```

```

main.tf > resource "aws_security_group" "cricbuzz_sg" > ingress
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.0"
6     }
7   }
8 }
9
10 provider "aws" {
11   region = "us-east-1"
12 }

```

Explanation

- terraform → Defines Terraform configuration.
- required_providers → Specifies which providers Terraform needs.
- aws → AWS provider.
- hashicorp/aws → Official AWS provider from HashiCorp.
- ~> 6.0 → Uses AWS provider version 6.x.
- provider "aws" → Configures AWS.
- region = "us-east-1" → Resources are created in the US East (N. Virginia) region.

Step 2: Create Security Group

The script creates an AWS Security Group for the EC2 instance.

```
resource "aws_security_group" "cricbuzz_sg" {
```

The Security Group allows incoming traffic on:

Port Purpose

22 SSH

8080 Nginx

8000 Docker application

SSH – Port 22

```

ingress {
  description = "SSH"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

```

This allows SSH connections to the EC2 instance.

Example:

```
ssh -i devops.key ubuntu@<PUBLIC-IP>
```

For a production environment, SSH should preferably be restricted to a trusted IP rather than 0.0.0.0/0.

Step 3: Allow Nginx Port 8080

```
ingress {  
    description = "Nginx 8080"  
    from_port   = 8080  
    to_port     = 8080  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

Port 8080 is opened because Nginx is configured to listen on port 8080.

Users access the application using:

```
http://<EC2-PUBLIC-IP>:8080
```

Step 4: Allow Docker Application Port 8000

```
ingress {  
    description = "Docker 8000"  
    from_port   = 8000  
    to_port     = 8000  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

This allows traffic to port 8000.

However, in this architecture, the external user normally accesses **Nginx on 8080**, while Nginx forwards the request internally to Docker on port 8000.

Step 5: Configure Outbound Traffic

```
egress {
```

```

from_port = 0
to_port   = 0
protocol  = "-1"
cidr_blocks = ["0.0.0.0/0"]
}

```

This allows the EC2 instance to make outbound connections to the internet.

This is required for things such as:

- apt-get update
- Installing packages
- Downloading Docker
- Cloning GitHub repository
- Downloading Go dependencies
- Pulling Docker images

```

14 # -----
15 # SECURITY GROUP
16 # -----
17
18 resource "aws_security_group" "cricbuzz_sg" {
19   name        = "cricbuzz-sg"
20   description = "Allow SSH and application traffic"
21   vpc_id      = "vpc-00148488443198382"
22
23   # SSH
24   ingress {
25     description = "SSH"
26     from_port   = 22
27     to_port     = 22
28     protocol    = "tcp"
29     cidr_blocks = ["0.0.0.0/0"]
30   }
31
32   # Nginx
33   ingress {
34     description = "Nginx 8080"
35     from_port   = 8080
36     to_port     = 8080
37     protocol    = "tcp"
38     cidr_blocks = ["0.0.0.0/0"]
39   }
40
41   # Docker application
42   ingress {
43     description = "Docker 8000"
44     from_port   = 8000
45     to_port     = 8000
46     protocol    = "tcp"
47     cidr_blocks = ["0.0.0.0/0"]
48   }
49
50   # Outbound
51   egress {
52     from_port   = 0
53     to_port     = 0
54     protocol    = "-1"
55     cidr_blocks = ["0.0.0.0/0"]
56   }
57
58   tags = {
59     Name = "Cricbuzz-SG"
60   }
61 }

```

Step 6: Create EC2 Instance

Terraform creates an EC2 instance using:

```
resource "aws_instance" "cricbuzz" {
```

Important configuration:

```
ami      = "ami-0b6d9d3d33ba97d99"
instance_type = "t3.micro"
key_name  = "devops.key"
subnet_id = "subnet-09d53b9b035621be1"
```

```
# -----
# EC2 INSTANCE
# -----

resource "aws_instance" "cricbuzz" {

  ami      = "ami-0b6d9d3d33ba97d99"
  instance_type = "t3.micro"

  key_name = "devops.key"

  subnet_id = "subnet-09d53b9b035621be1"

  vpc_security_group_ids = [
    aws_security_group.cricbuzz_sg.id
  ]
}
```

Explanation

- ami → Ubuntu machine image.
- instance_type → t3.micro.
- key_name → Existing EC2 key pair used for SSH.
- subnet_id → Subnet where EC2 is launched.

Step 7: Attach Security Group to EC2

```
vpc_security_group_ids = [
  aws_security_group.cricbuzz_sg.id
]
```

This connects the Security Group created by Terraform to the EC2 instance.

So the EC2 instance receives the inbound/outbound rules defined earlier.

Step 8: Use User Data for Automatic Configuration

The most important part of the script is:

```
user_data = <<-EOF
```

The User Data script runs automatically when the EC2 instance is launched.

Instead of manually logging into the server and installing everything, Terraform automatically performs the setup.

The overall process is:

EC2 Created



User Data Starts



Update OS



Install Packages



Install Docker



Install Nginx



Clone Git Repository



Create Dockerfile



Create Docker Compose



Build Docker Image



Start Container



Configure Nginx



Configure Logrotate



Run Tests

Step 9: Enable Error Handling and Logging

The script starts with:

set -euxo pipefail

This helps identify errors during the User Data execution.

The script also saves the bootstrap output:

```
exec > >(tee -a /var/log/user-data.log | logger -t user-data -s 2>/dev/console) 2>&1
```

Therefore, if the deployment fails, we can check:

```
cat /var/log/user-data.log
```

This is useful for troubleshooting EC2 bootstrap issues.

```
user_data = <<-EOF
#!/bin/bash

set -euxo pipefail

# Save complete bootstrap log
exec > >(tee -a /var/log/user-data.log | logger -t user-data -s 2>/dev/console) 2>&1

echo "===== STARTING CRICBUZZ SETUP ====="
```

Step 10: Update Ubuntu

The script runs:

```
apt-get update -y
```

```
apt-get upgrade -y
```

Purpose

```
apt-get update
```

→ Updates the package repository information.

```
apt-get upgrade
```

→ Upgrades installed packages to newer versions.

```
# -----
# SYSTEM UPDATE
# -----

apt-get update -y
apt-get upgrade -y
```

Step 11: Install Required Packages

The script installs:

```
apt-get install -y \
```

```
ca-certificates \
```

```
curl \
```

```
gnupg \
```

```
git \
```

nginx \

logrotate

Packages

Package	Purpose
ca-certificates	SSL/TLS certificates
curl	Download/test URLs
gnupg	Verify package repository signatures
git	Clone application source code
nginx	Web server/reverse proxy
logrotate	Rotate application/server logs

```
# -----  
# INSTALL BASIC PACKAGES  
# -----  
  
apt-get install -y \  
ca-certificates \  
curl \  
gnupg \  
git \  
nginx \  
logrotate
```

Step 12: Install Docker

The script creates the Docker keyring directory:

```
install -m 0755 -d /etc/apt/keyrings
```

Then downloads Docker's GPG key:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \  
-o /etc/apt/keyrings/docker.asc
```

Then it adds the official Docker repository.

After that:

```
apt-get update -y
```

The following Docker components are installed:

```
apt-get install -y \  
docker-ce \  
docker-ce-cli \
```

containerd.io \

docker-buildx-plugin \

docker-compose-plugin

```
# -----  
# INSTALL DOCKER FROM OFFICIAL DOCKER REPOSITORY  
# -----  
  
install -m 0755 -d /etc/apt/keyrings  
  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \  
-o /etc/apt/keyrings/docker.asc  
  
chmod a+r /etc/apt/keyrings/docker.asc  
  
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" \  
> /etc/apt/sources.list.d/docker.list  
  
apt-get update -y  
  
apt-get install -y \  
docker-ce \  
docker-ce-cli \  
containerd.io \  
docker-buildx-plugin \  
docker-compose-plugin
```

Components

- docker-ce → Docker Engine
- docker-ce-cli → Docker command-line interface
- containerd.io → Container runtime
- docker-buildx-plugin → Advanced Docker image building
- docker-compose-plugin → Docker Compose

Step 13: Start and Enable Docker

systemctl enable docker

systemctl start docker

Meaning

enable

→ Starts Docker automatically when the server boots.

start

→ Starts Docker immediately.

The script verifies the installation:

docker --version

docker compose version

Step 14: Start Nginx

```
systemctl enable nginx
```

```
systemctl start nginx
```

Nginx is:

- Enabled at boot
- Started immediately

Later, Nginx will act as a **reverse proxy** in front of the Docker application.

```
systemctl enable docker
systemctl start docker

systemctl enable nginx
systemctl start nginx

echo "===== DOCKER INSTALLED ====="

docker --version
docker compose version
```

Step 15: Clone the Application Repository

The script removes any previous application directory:

```
rm -rf /opt/cricbuzz-docker-hello-world
```

Then clones the GitHub repository:

```
git clone \
  https://github.com/betawins/cricbuzz-docker-hello-world.git \
  /opt/cricbuzz-docker-hello-world
```

The application is therefore stored at:

```
/opt/cricbuzz-docker-hello-world
```

Then the script enters the directory:

```
cd /opt/cricbuzz-docker-hello-world
```

```
# -----
# CLONE APPLICATION
# -----

rm -rf /opt/cricbuzz-docker-hello-world

git clone \
  https://github.com/betawins/cricbuzz-docker-hello-world.git \
  /opt/cricbuzz-docker-hello-world

cd /opt/cricbuzz-docker-hello-world
```

Step 16: Create a Multi-Stage Dockerfile

The script creates:

Dockerfile.terraform

The Dockerfile contains two stages.

```
# -----  
# CREATE A RELIABLE MULTI-STAGE DOCKERFILE  
# -----  
#  
# The original repository Dockerfile expects a pre-built  
# "hello_world" binary.  
#  
# We build that binary INSIDE Docker instead.  
# -----  
  
cat > Dockerfile.terraform <<'DOCKERFILE'
```

Stage 1 – Build

FROM golang:1.24-bookworm AS builder

This stage provides the Go environment required to compile the application.

The source code and Go dependencies are copied:

COPY go.mod go.sum ./

RUN go mod download

COPY . .

Then the application is compiled:

RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
go build -buildvcs=false -o hello_world .

The result is a binary called:

hello_world

```
# -----  
# BUILD STAGE  
# -----  
FROM golang:1.24-bookworm AS builder  
  
WORKDIR /app  
  
COPY go.mod go.sum ./  
  
RUN go mod download  
  
COPY . .  
  
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \  
go build -buildvcs=false -o hello_world .
```

Step 17: Create Final Docker Image

The second stage uses:

```
FROM ghcr.io/vincetse/scratch:latest
```

Only the compiled application is copied:

```
COPY --from=builder /app/hello_world /hello_world
```

Then:

```
ENV GIN_MODE=release
```

sets the Gin framework to release mode.

The application exposes:

```
EXPOSE 8080
```

And starts using:

```
ENTRYPOINT ["/hello_world"]
```

```
# FINAL STAGE
# -----
FROM ghcr.io/vincetse/scratch:latest

COPY --from=builder /app/hello_world /hello_world

ENV GIN_MODE=release

EXPOSE 8080

ENTRYPOINT ["/hello_world"]

DOCKERFILE
```

Why Multi-Stage Build?

The Go compiler and build dependencies are needed only during the build stage.

The final image contains only what is required to run the application.

Step 18: Create Docker Compose Configuration

The script creates:

```
docker-compose.yml
```

The service is:

```
services:
```

```
  cb-app:
```

The image is built using:

```
build:
```

```
  context: .
```

dockerfile: Dockerfile.terraform

The container is named:

container_name: cb-app

```
# -----  
# CREATE DOCKER COMPOSE  
# -----  
#  
# Host port 8000 -> Container port 8080  
# The Go application listens on 8080.  
# -----  
  
cat > docker-compose.yml <<'COMPOSE'  
  
services:  
  cb-app:  
    build:  
      context: .  
      dockerfile: Dockerfile.terraform  
    container_name: cb-app
```

Step 19: Configure Port Mapping

This is an important concept:

ports:

- "8000:8080"

The format is:

HOST PORT : CONTAINER PORT

Therefore:

EC2 Host

Port 8000

↓

Docker Container

Port 8080

The Go application listens on **8080 inside the container**.

Docker makes it accessible on **8000 on the EC2 host**.

Step 20: Configure Container Restart Policy

restart: always

This tells Docker to automatically restart the container if it stops.

It also helps the application come back after Docker/host restarts.

```
ports:
  - "8000:8080"
restart: always
```

Step 21: Build and Start Application

The script executes:

```
docker compose up -d --build
```

Meaning

up

→ Creates and starts the service.

-d

→ Runs in detached/background mode.

--build

→ Builds the Docker image before starting the container.

So the application is automatically built and started.

```
# -----
# BUILD AND START APPLICATION
# -----

docker compose up -d --build
```

Step 22: Verify Docker Container

The script waits:

```
sleep 10
```

Then checks:

```
docker compose ps
```

This verifies whether the container is running.

You can also check:

```
docker ps
```

Expected architecture:

```
cb-app
```

|

└─ Container Port 8080

↑

|

Host Port 8000

```
# WAIT FOR CONTAINER
# -----

sleep 10

docker compose ps
```

Step 23: Configure Nginx as Reverse Proxy

The script creates an Nginx configuration:

/etc/nginx/sites-available/cricbuzz

Nginx listens on:

listen 8080;

Then it forwards requests to:

proxy_pass http://127.0.0.1:8000;

So the request flow becomes:

Browser

|

| :8080

↓

Nginx

|

| :8000

↓

Docker Container

|

| :8080

↓

Cricbuzz Application

```
# -----  
# NGINX REVERSE PROXY  
# 8080 -> 8000  
# -----  
  
cat > /etc/nginx/sites-available/cricbuzz <<'NGINX'  
  
server {  
    listen 8080;  
    listen [::]:8080;  
  
    access_log /var/log/cb.log;  
    error_log /var/log/cb.log;  
  
    location / {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_http_version 1.1;
```

Step 24: Configure Proxy Headers

The configuration contains:

```
proxy_set_header Host $host;
```

```
proxy_set_header X-Real-IP $remote_addr;
```

```
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
proxy_set_header X-Forwarded-Proto $scheme;
```

These headers pass useful information from the client request to the backend application.

For example:

- Original host
- Client IP
- Forwarded IP information
- HTTP/HTTPS protocol

Step 25: Enable Nginx Configuration

The default Nginx configuration is removed:

```
rm -f /etc/nginx/sites-enabled/default
```

Then the Cricbuzz configuration is enabled:

```
ln -sf \
```

```
/etc/nginx/sites-available/cricbuzz \
```

```
/etc/nginx/sites-enabled/cricbuzz
```

This creates a symbolic link from:

sites-available



sites-enabled

```
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

Step 26: Test Nginx Configuration

Before restarting Nginx, the script runs:

nginx -t

This checks whether the Nginx configuration syntax is valid.

If the configuration is valid, Nginx is restarted:

systemctl restart nginx

```
NGINX
# -----
# ENABLE NGINX CONFIG
# -----

rm -f /etc/nginx/sites-enabled/default

ln -sf \
    /etc/nginx/sites-available/cricbuzz \
    /etc/nginx/sites-enabled/cricbuzz

# Test configuration
nginx -t

# Restart Nginx
systemctl restart nginx
```

Step 27: Configure Application Logging

The script creates:

touch /var/log/cb.log

The file is given permissions:

chmod 644 /var/log/cb.log

Nginx is configured to write:

access_log /var/log/cb.log;

```
error_log /var/log/cb.log;
```

Therefore, application proxy access and error information can be stored in:

`/var/log/cb.log`

```
# -----  
# LOG FILE  
# -----  
  
touch /var/log/cb.log  
  
chmod 644 /var/log/cb.log
```

Step 28: Configure Logrotate

The script creates:

`/etc/logrotate.d/cb`

It configures:

`daily`

`rotate 7`

`compress`

`delaycompress`

Meaning

- `daily` → Rotate logs every day.
- `rotate 7` → Keep 7 rotated log files.
- `compress` → Compress old logs.
- `delaycompress` → Compress from the second-oldest rotation.
- `missingok` → Don't report an error if the log is missing.
- `notifempty` → Don't rotate an empty log.

This prevents log files from continuously consuming disk space.

```
# -----  
# LOGROTATE  
# -----  
  
cat > /etc/logrotate.d/cb <<'LOGROTATE'  
  
/var/log/cb.log {  
    daily  
    rotate 7  
    compress  
    delaycompress  
    missingok  
    notifempty  
    create 0644 root root  
    sharedscripts  
    postrotate  
        systemctl reload nginx > /dev/null 2>&1 || true  
    endscript  
}
```

Step 29: Final Docker Verification

The script executes:

```
docker ps
```

This displays running containers.

It confirms that:

```
cb-app
```

is running.

Step 30: Check Listening Ports

The script runs:

```
ss -ltnp
```

This displays TCP ports currently listening on the server.

We expect to see services related to:

8080 → Nginx

8000 → Docker application

Step 31: Test Docker Application

The script tests:

```
curl -f http://localhost:8000/ || true
```

This verifies that the application is reachable through the Docker host port.

Flow:

localhost:8000



Docker



Container:8080



Cricbuzz application

Step 32: Test Nginx

The script also tests:

```
curl -f http://localhost:8080/ || true
```

This verifies:

localhost:8080



Nginx



localhost:8000



Docker



Application

```
# -----  
# FINAL CHECKS  
# -----  
  
echo "===== DOCKER CONTAINERS ====="  
  
docker ps  
  
echo "===== LISTENING PORTS ====="  
  
ss -lntp  
  
echo "===== LOCAL DOCKER TEST ====="  
  
curl -f http://localhost:8000/ || true  
  
echo "===== LOCAL NGINX TEST ====="  
  
curl -f http://localhost:8080/ || true  
  
echo "===== SETUP COMPLETE ====="  
  
EOF  
}
```

Step 33: Terraform Outputs

Finally, Terraform provides three outputs.

Instance ID

```
output "instance_id" {  
    value = aws_instance.cricbuzz.id  
}
```

This displays the EC2 instance ID.

Example:

i-0123456789abcdef

Public IP

```
output "public_ip" {  
    value = aws_instance.cricbuzz.public_ip  
}
```

This displays the EC2 public IP.

Application URL

```
output "application_url" {  
    value = "http://${aws_instance.cricbuzz.public_ip}:8080"  
}
```

This gives the final application URL:

http://<PUBLIC-IP>:8080

```
# -----  
# OUTPUTS  
# -----  
  
output "instance_id" {  
  value = aws_instance.cricbuzz.id  
}  
  
output "public_ip" {  
  value = aws_instance.cricbuzz.public_ip  
}  
  
output "application_url" {  
  value = "http://${aws_instance.cricbuzz.public_ip}:8080"  
}
```

FINAL RESULT:

```
Do you want to perform these actions?  
  Terraform will perform the actions described above.  
  Only 'yes' will be accepted to approve.  
  
Enter a value: yes  
  
aws_security_group.cricbuzz_sg: Creating...  
aws_security_group.cricbuzz_sg: Creation complete after 6s [id=sg-0d497fd66c1195e16]  
aws_instance.cricbuzz: Creating...  
aws_instance.cricbuzz: Still creating... [00m10s elapsed]  
aws_instance.cricbuzz: Creation complete after 16s [id=i-0844a6c581e78edae]  
  
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
application_url = "http://13.222.101.211:8080"  
instance_id = "i-0844a6c581e78edae"  
public_ip = "13.222.101.211"  
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform plan  
aws_security_group.cricbuzz_sg: Refreshing state... [id=sg-0d497fd66c1195e16]  
aws_instance.cricbuzz: Refreshing state... [id=i-0844a6c581e78edae]  
  
No changes. Your infrastructure matches the configuration.
```

```
ubuntu@ip-172-31-4-197:~$ sudo systemctl status nginx --no-pager  
● nginx.service - A high performance web server and a reverse proxy server  
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)  
   Active: active (running) since Tue 2026-09-15 15:53:28 UTC; 19s ago  
  Invocation: 66e22bfb0114a73b9cf2645739843f5  
     Docs: man:nginx(8)  
   Main PID: 25834 (nginx)  
     Tasks: 3 (limit: 627)  
    Memory: 3M (peak: 7.1M)  
       CPU: 45ms  
    CGroup: /system.slice/nginx.service  
            └─25834 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"  
              └─25837 "nginx: worker process"  
                └─25838 "nginx: worker process"
```

```
ubuntu@ip-172-31-4-197:~$ sudo docker ps  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES  
de3b57e1ae49   cricbuzz-docker-hello-world-cb-app   "/hello world"    6 minutes ago   Up 6 minutes   0.0.0.0:8000->8080/tcp, [::]:8000->8080/tcp   cb-app  
ubuntu@ip-172-31-4-197:~$ sudo ss -lntp | grep -E '8000|8080'  
LISTEN 0      4096      0.0.0.0:8000      0.0.0.0:*      users: (("docker-proxy",pid=29490,fd=8))  
LISTEN 0      511      0.0.0.0:8080     0.0.0.0:*      users: (("nginx",pid=29549,fd=6), ("nginx",pid=29548,fd=6), ("nginx",pid=29547,fd=6))  
LISTEN 0      4096      [::]:8000       [::]:*        users: (("docker-proxy",pid=29496,fd=8))  
LISTEN 0      511      [::]:8080       [::]:*        users: (("nginx",pid=29549,fd=7), ("nginx",pid=29548,fd=7), ("nginx",pid=29547,fd=7))  
ubuntu@ip-172-31-4-197:~$ curl http://localhost:8000/  
Hello, World!  
ubuntu@ip-172-31-4-197:~$
```

< Not secure 13.222.101.211:8080

Hello, World!

```

ubuntu@ip-172-31-4-197:~$ curl http://localhost:8000/
Hello, World!ubuntu@ip-172-31-4-197:~$ curl http://localhost:8080/
Hello, World!ubuntu@ip-172-31-4-197:~$ sudo cat /var/log/cb.log
::1 - - [15/Sep/2026:15:56:02 +0000] "GET / HTTP/1.1" 200 13 "-" "curl/8.18.0"
103.203.172.230 - - [15/Sep/2026:16:00:46 +0000] "GET / HTTP/1.1" 200 13 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
536 Edg/153.0.0.0"
103.203.172.230 - - [15/Sep/2026:16:00:46 +0000] "GET /favicon.ico HTTP/1.1" 404 18 "http://13.222.101.211:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebK
ke Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0"
::1 - - [15/Sep/2026:16:07:44 +0000] "GET / HTTP/1.1" 200 13 "-" "curl/8.18.0"
ubuntu@ip-172-31-4-197:~$ cd /etc/logrotate.d/
ubuntu@ip-172-31-4-197:/etc/logrotate.d$ ll
total 72
drwxr-xr-x 2 root root 4096 Sep 15 15:56 ./
drwxr-xr-x 111 root root 4096 Sep 15 15:53 ../
-rw-r--r-- 1 root root 120 Oct 8 2025 alternatives
-rw-r--r-- 1 root root 126 Nov 12 2024 apport
-rw-r--r-- 1 root root 173 Apr 7 09:02 apt
-rw-r--r-- 1 root root 91 Aug 31 2025 bootlog
-rw-r--r-- 1 root root 130 Jun 22 2020 btmap
-rw-r--r-- 1 root root 244 Sep 15 15:56 cb
ubuntu@ip-172-31-4-197:/etc/logrotate.d$ cat cb
/var/log/cb.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0644 root root
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null 2>&1 || true
    endscript
}
ubuntu@ip-172-31-4-197:/etc/logrotate.d$

```

I WRITE THIS BELOW SCRIPT FOR TASK:

```
terraform {
```

```
    required_providers {
```

```
        aws = {
```

```
            source = "hashicorp/aws"
```

```
            version = "~> 6.0"
```

```
        }
```

```
    }
```

```
}
```

```
provider "aws" {
```

```
    region = "us-east-1"
```

```
}
```

```
# -----
```

```
# SECURITY GROUP
```

```
# -----
```

```
resource "aws_security_group" "cricbuzz_sg" {  
  name      = "cricbuzz-sg"  
  description = "Allow SSH and application traffic"  
  vpc_id    = "vpc-00148488443198382"
```

```
# SSH
```

```
ingress {  
  description = "SSH"  
  from_port   = 22  
  to_port     = 22  
  protocol    = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

```
# Nginx
```

```
ingress {  
  description = "Nginx 8080"  
  from_port   = 8080  
  to_port     = 8080  
  protocol    = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

```
# Docker application
```

```
ingress {  
  description = "Docker 8000"  
  from_port   = 8000  
  to_port     = 8000  
  protocol    = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

```
# Outbound
```

```
egress {  
    from_port = 0  
    to_port   = 0  
    protocol  = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

```
tags = {  
    Name = "Cricbuzz-SG"  
}  
}
```

```
# -----  
# EC2 INSTANCE  
# -----
```

```
resource "aws_instance" "cricbuzz" {  
  
    ami          = "ami-0b6d9d3d33ba97d99"  
    instance_type = "t3.micro"  
  
    key_name = "devops.key"  
  
    subnet_id = "subnet-09d53b9b035621be1"  
  
    vpc_security_group_ids = [  
        aws_security_group.cricbuzz_sg.id  
    ]  
}
```

```
# -----
```

```
# USER DATA
```

```
# -----
```

```
user_data = <<-EOF
```

```
#!/bin/bash
```

```
set -euxo pipefail
```

```
# Save complete bootstrap log
```

```
exec >>(tee -a /var/log/user-data.log | logger -t user-data -s 2>/dev/console) 2>&1
```

```
echo "===== STARTING CRICBUZZ SETUP ====="
```

```
# -----
```

```
# SYSTEM UPDATE
```

```
# -----
```

```
apt-get update -y
```

```
apt-get upgrade -y
```

```
# -----
```

```
# INSTALL BASIC PACKAGES
```

```
# -----
```

```
apt-get install -y \
```

```
ca-certificates \
```

```
curl \
```

```
gnupg \
```

```
git \
```

```
nginx \
```

logrotate

INSTALL DOCKER FROM OFFICIAL DOCKER REPOSITORY

install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc

echo \

"deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \

\$(. /etc/os-release && echo "\$VERSION_CODENAME") stable" \
> /etc/apt/sources.list.d/docker.list

apt-get update -y

apt-get install -y \

docker-ce \

docker-ce-cli \

containerd.io \

docker-buildx-plugin \

docker-compose-plugin

systemctl enable docker

systemctl start docker

```
systemctl enable nginx
```

```
systemctl start nginx
```

```
echo "===== DOCKER INSTALLED ====="
```

```
docker --version
```

```
docker compose version
```

```
# -----
```

```
# CLONE APPLICATION
```

```
# -----
```

```
rm -rf /opt/cricbuzz-docker-hello-world
```

```
git clone \
```

```
https://github.com/betawins/cricbuzz-docker-hello-world.git \
```

```
/opt/cricbuzz-docker-hello-world
```

```
cd /opt/cricbuzz-docker-hello-world
```

```
# -----
```

```
# CREATE A RELIABLE MULTI-STAGE DOCKERFILE
```

```
# -----
```

```
#
```

```
# The original repository Dockerfile expects a pre-built
```

```
# "hello_world" binary.
```

```
#
```

```
# We build that binary INSIDE Docker instead.
```

```
# -----
```

```
cat > Dockerfile.terraform <<'DOCKERFILE'
```

```
# -----  
  
# BUILD STAGE  
  
# -----  
  
FROM golang:1.24-bookworm AS builder  
  
WORKDIR /app  
  
COPY go.mod go.sum ./  
  
RUN go mod download  
  
COPY . .  
  
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \  
    go build -buildvcs=false -o hello_world .  
  
# -----  
  
# FINAL STAGE  
  
# -----  
  
FROM ghcr.io/vincetse/scratch:latest  
  
COPY --from=builder /app/hello_world /hello_world  
  
ENV GIN_MODE=release  
  
EXPOSE 8080  
  
ENTRYPOINT ["/hello_world"]  
  
DOCKERFILE
```

```
# -----  
# CREATE DOCKER COMPOSE  
# -----  
#  
# Host port 8000 -> Container port 8080  
# The Go application listens on 8080.  
# -----
```

```
cat > docker-compose.yml <<'COMPOSE'
```

```
services:
```

```
  cb-app:
```

```
    build:
```

```
      context: .
```

```
      dockerfile: Dockerfile.terraform
```

```
    container_name: cb-app
```

```
    ports:
```

```
      - "8000:8080"
```

```
    restart: always
```

```
COMPOSE
```

```
# -----  
# BUILD AND START APPLICATION  
# -----
```

```
docker compose up -d --build
```

```
# -----
```

```
# WAIT FOR CONTAINER
```

```
# -----
```

```
sleep 10
```

```
docker compose ps
```

```
# -----
```

```
# NGINX REVERSE PROXY
```

```
# 8080 -> 8000
```

```
# -----
```

```
cat > /etc/nginx/sites-available/cricbuzz <<'NGINX'
```

```
server {
```

```
    listen 8080;
```

```
    listen [::]:8080;
```

```
    access_log /var/log/cb.log;
```

```
    error_log /var/log/cb.log;
```

```
    location / {
```

```
        proxy_pass http://127.0.0.1:8000;
```

```
        proxy_http_version 1.1;
```

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;

}
}
```

NGINX

```
# -----
# ENABLE NGINX CONFIG
# -----
```

```
rm -f /etc/nginx/sites-enabled/default
```

```
ln -sf \
/etc/nginx/sites-available/cricbuzz \
/etc/nginx/sites-enabled/cricbuzz
```

```
# Test configuration
```

```
nginx -t
```

```
# Restart Nginx
```

```
systemctl restart nginx
```

```
# -----
# LOG FILE
# -----
```

```
touch /var/log/cb.log
```

```
chmod 644 /var/log/cb.log
```

```
# -----
```

```
# LOGROTATE
```

```
# -----
```

```
cat > /etc/logrotate.d/cb <<'LOGROTATE'
```

```
/var/log/cb.log {
```

```
    daily
```

```
    rotate 7
```

```
    compress
```

```
    delaycompress
```

```
    missingok
```

```
    notifempty
```

```
    create 0644 root root
```

```
    sharedscripts
```

```
    postrotate
```

```
        systemctl reload nginx > /dev/null 2>&1 || true
```

```
endscript
```

```
}
```

```
LOGROTATE
```

```
# -----
```

```
# FINAL CHECKS
```

```
# -----
```

```
echo "===== DOCKER CONTAINERS ====="
```

```
docker ps
```

```
echo "===== LISTENING PORTS ====="
```

```
ss -lntp
```

```
echo "===== LOCAL DOCKER TEST ====="
```

```
curl -f http://localhost:8000/ || true
```

```
echo "===== LOCAL NGINX TEST ====="
```

```
curl -f http://localhost:8080/ || true
```

```
echo "===== SETUP COMPLETE ====="
```

```
EOF
```

```
}
```

```
# -----
```

```
# OUTPUTS
```

```
# -----
```

```
output "instance_id" {  
    value = aws_instance.cricbuzz.id  
}
```

```
output "public_ip" {  
    value = aws_instance.cricbuzz.public_ip  
}
```

```
output "application_url" {  
    value = "http://${aws_instance.cricbuzz.public_ip}:8080"  
}
```

Complete Deployment Flow:

